WEEK 6 · PSDF GenAI Batch 2026

Monday — Week 6

Hybrid Search: BM25 + Dense Vector

Why Vector-Only Fails · BM25 Explained · RRF Fusion · LangChain EnsembleRetriever

By the end of today you will be able to:
Explain why pure vector search fails on keyword-heavy and exact-match queries
Describe how BM25 keyword retrieval works and what it does better than embeddings
Implement hybrid search by combining BM25 and dense vector results using RRF
Use LangChain EnsembleRetriever to build hybrid search in 10 lines of code
Measure retrieval quality before and after adding BM25 using recall@5
Decide when to use pure vector, pure keyword, or hybrid search for a given use case

NOTE	Week 5 recap: 6-stage RAG pipeline (load→chunk→embed→store→retrieve→generate). RecursiveCharacterTextSplitter. ChromaDB persistence. Retrieval debugging (missing context, wrong chunk, context too long). RAGAS metrics preview. This week: upgrade retrieval quality significantly.

June 8, 2026

Online (Zoom) · 10:00 AM – 1:00 PM · Mon–Thu | In-Person Friday 10:00 AM – 1:00 PM
Crewlogix — The Institute of Technology · Gulberg III, Lahore

crewlogix
The Institute of Technology

Session — Monday June 8	3 hours	■ Online — Zoom · 10:00 AM – 1:00 PM

Topic 1 — The Weakness of Pure Vector Search (25 min)

Vector search is powerful for semantic similarity — finding documents that mean the same thing regardless of exact words. But it fails in predictable ways that hurt real-world RAG applications.

1.1 Three cases where vector search fails

Failure case	Example	Why vector search fails
Exact product code or ID	User asks: "status of complaint #PKR-2024-00892"	An embedding model has never seen this specific ID. It maps to a generic vector near "complaint status" — matches wrong documents.
Rare technical acronyms	User asks: "SECP Form 29 requirements"	"Form 29" is an exact regulatory term. The embedding maps it near "form" and "requirements" — misses the specific regulation.
Names and proper nouns	User asks: "Asad Umar's statement on AI policy"	The name vector is noisy — matches other politicians. A keyword search finds the exact name instantly.
Numerical values	User asks: "PKR 45,000 training fee"	Numbers are poorly represented in embedding space. Keyword match on "45,000" is exact and instant.

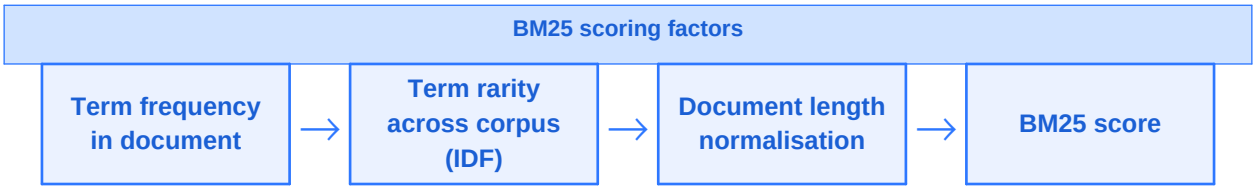
The pattern is clear: vector search excels at **semantic similarity** ("bills" matches "invoices"). Keyword search excels at **exact match** (IDs, codes, names, numbers). A hybrid system uses both — getting semantic recall AND exact precision.

PAKISTAN EG	A Pakistani legal chatbot indexed 5,000 SECP regulation documents. A user asks "Section 47(2) penalties". Vector search returns documents about penalties in general — not Section 47(2) specifically. BM25 finds "Section 47(2)" as an exact string match in milliseconds. Hybrid search returns the exact section at the top.

Topic 2 — BM25: The Best Keyword Retrieval Algorithm (40 min)

BM25 (Best Matching 25) is the gold standard keyword search algorithm — used by Elasticsearch, Solr, and most major search engines. It is more sophisticated than simple keyword counting: it accounts for term frequency, document length, and how rare a term is across the entire corpus.

2.1 How BM25 scores a document



BM25 factor	What it measures	Effect on score
Term Frequency (TF)	How many times the query term appears in the document	More occurrences = higher score, but with diminishing returns (not linear)
Inverse Document Frequency (IDF)	How rare the term is across ALL documents in the collection	Rare terms score higher — "SECP" in a collection with 1 SECP doc beats "the" anywhere
Document length normalisation	Penalises long documents that match just because they are long	A 10-word doc with 3 matches outscores a 1,000-word doc with 3 matches

Plain English: BM25 rewards documents where the query terms are specific (rare in the corpus), appear multiple times, and the document is not suspiciously long. This is exactly what you want for exact-match retrieval.

2.2 BM25 vs TF-IDF vs Vector Search — choosing the right tool

Method	Best for	Fails on	Speed
BM25 (keyword)	Exact terms: IDs, codes, names, technical terms, numbers	Semantic similarity — "purchase" misses "buy"	Very fast — inverted index lookup
TF-IDF (older keyword)	Simple keyword matching	Same as BM25 but less accurate on longer documents	Fast
Dense vector (embedding)	Semantic similarity — synonyms, paraphrases, related concepts	Exact match — IDs, rare terms, proper nouns	Moderate — requires embedding the query
Hybrid (BM25 + vector)	Both exact match AND semantic similarity	Very little — best of both worlds	Slightly slower but worth it

EXAM TIP

AWS exam: "A developer notices their RAG system retrieves semantically similar documents but misses documents containing exact product codes. What retrieval improvement would help?" Answer: Hybrid search combining dense vector retrieval with BM25 keyword retrieval. This is directly tested in AWS RAG optimisation content.

2.3 BM25 in Python — rank_bm25 library

```
# Install
!pip install rank_bm25 -q

from rank_bm25 import BM25Okapi
```

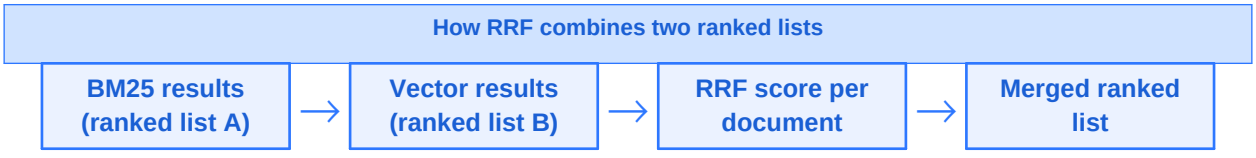
```
import nltk
nltk.download("punkt", quiet=True)
from nltk.tokenize import word_tokenize

# Sample Pakistani regulatory documents
documents = [
 "SECP Form 29 must be filed within 14 days of director appointment.",
 "Section 47 penalties for late filing range from PKR 5,000 to 50,000.",
 "NTN registration is mandatory for all companies before tax filing.",
 "Foreign company registration requires Form S-47 and audited accounts.",
 "Annual return under Section 130 must be filed by December 31.",
]

# Tokenise: BM25 works on word tokens, not raw strings
tokenised = [word_tokenize(doc.lower()) for doc in documents]
bm25 = BM25Okapi(tokenised)

# Query
query = "Form 29 director filing"
query_tokens = word_tokenize(query.lower())
scores = bm25.get_scores(query_tokens)

ranked = sorted(zip(scores, documents), reverse=True)
for score, doc in ranked[:3]:
 print(f"Score: {score:.4f} | {doc[:70]}")
# Score: 4.2341 | SECP Form 29 must be filed within 14 days of director...
# Score: 1.1823 | Foreign company registration requires Form S-47...
# Score: 0.0000 | NTN registration is mandatory for all companies...
```

BM25 correctly surfaces the Form 29 document at the top because "Form" and "29" are both present and "Form 29" is a relatively rare phrase in this corpus. The NTN document scores zero — it shares no query terms at all.

Topic 3 — Reciprocal Rank Fusion (RRF): Combining Two Result Lists (35 min)

When you have two ranked lists of results — one from BM25 and one from vector search — how do you combine them into a single, better-ranked list? The standard answer is **Reciprocal Rank Fusion (RRF)**.

3.1 The RRF formula



For each document, RRF computes a score based on its rank in each list:

```
# RRF score for a document
# k = 60 (standard constant that dampens the effect of very top ranks)
rrf_score = 1/(k + rank_in_list_A) + 1/(k + rank_in_list_B)
```

A document ranked 1st in BM25 and 3rd in vector search scores higher than one ranked 5th in both. A document absent from one list gets a score of 0 for that list — it can still score well if it ranks highly in the other.

> **NOTE** RRF uses ranks, not raw scores. This is important: BM25 scores (e.g. 4.2) and cosine similarity scores (e.g. 0.84) are on completely different scales and cannot be averaged directly. RRF sidesteps this by only using the rank position.

3.2 RRF implementation in Python

```
# Manual RRF implementation — understand the logic before using LangChain
def rrf_fusion(bm25_docs, vector_docs, k=60):
 """
 Combine two ranked lists using Reciprocal Rank Fusion.
 bm25_docs: list of doc strings, ranked by BM25 (index 0 = rank 1)
 vector_docs: list of doc strings, ranked by vector search
 Returns: merged list sorted by RRF score (highest first)
 """
 scores = {}

 for rank, doc in enumerate(bm25_docs, start=1):
 scores[doc] = scores.get(doc, 0) + 1 / (k + rank)

 for rank, doc in enumerate(vector_docs, start=1):
 scores[doc] = scores.get(doc, 0) + 1 / (k + rank)

 return sorted(scores.keys(), key=lambda d: scores[d], reverse=True)

# Example usage
bm25_results = ["doc_A", "doc_C", "doc_B", "doc_E", "doc_D"]
vector_results = ["doc_C", "doc_A", "doc_D", "doc_B", "doc_F"]

merged = rrf_fusion(bm25_results, vector_results)
print("Merged ranking:", merged)
# Merged: ["doc_A", "doc_C", "doc_B", "doc_D", "doc_E", "doc_F"]
```

doc_A (rank 1 in BM25, rank 2 in vector) and doc_C (rank 2 in BM25, rank 1 in vector) both rise to the top. doc_F, only appearing in vector search, still makes the merged list — RRF does not penalise documents that appear in only one system.

Topic 4 — LangChain EnsembleRetriever (40 min)

LangChain's EnsembleRetriever implements RRF fusion out of the box. You provide two retrievers — one BM25, one dense vector — with optional weights, and it handles everything else.

4.1 Setup: both retrievers

```
# Install required packages
!pip install langchain langchain-community rank_bm25 chromadb sentence-transformers -q

from langchain_community.retrievers import BM25Retriever
from langchain_community.vectorstores import Chroma
from langchain.retrievers import EnsembleRetriever
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.schema import Document

# Our Pakistani regulatory document set (as LangChain Document objects)
docs = [
 Document(page_content="SECP Form 29 must be filed within 14 days.",
 metadata={"source": "secp", "section": "directors"}),
 Document(page_content="Section 47 penalties range from PKR 5,000 to 50,000.",
 metadata={"source": "secp", "section": "penalties"}),
 Document(page_content="NTN registration mandatory before any tax filing.",
 metadata={"source": "fbr", "section": "registration"}),
 Document(page_content="Foreign company registration requires audited accounts.",
 metadata={"source": "secp", "section": "foreign"}),
 Document(page_content="Annual return under Section 130 by December 31.",
 metadata={"source": "secp", "section": "annual"}),
]
```

4.2 Build the BM25 retriever

```
# BM25 retriever — works directly on Document objects
bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = 3 # return top-3 results
```

4.3 Build the vector retriever

```
# Dense vector retriever using ChromaDB + HuggingFace embeddings
embedding_model = HuggingFaceEmbeddings(
 model_name="sentence-transformers/all-MiniLM-L6-v2")

vectorstore = Chroma.from_documents(docs, embedding_model)
vector_retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

4.4 Combine with EnsembleRetriever

```
# Ensemble: 50% weight to BM25, 50% to vector — adjust based on your use case
ensemble_retriever = EnsembleRetriever(
```

```
 retrievers = [bm25_retriever, vector_retriever],
 weights = [0.5, 0.5]
)

# Query — retrieves from both and fuses with RRF automatically
query = "Form 29 director appointment requirements"
results = ensemble_retriever.get_relevant_documents(query)

for i, doc in enumerate(results, 1):
 print(f"\n[{i}] Source: {doc.metadata['source']} | {doc.page_content[:70]}")
# [1] Source: secp | SECP Form 29 must be filed within 14 days...
# [2] Source: secp | Foreign company registration requires audited...
# [3] Source: secp | Section 47 penalties range from PKR 5,000...
```

The EnsembleRetriever correctly surfaces the Form 29 document at position 1 — driven by BM25 precision on "Form 29" — while also including semantically related documents from vector search.

4.5 Tuning the weights

Use case	Recommended weights (BM25 : Vector)	Reasoning
General Q&A; on prose documents	0.3 : 0.7	Most queries are semantic — weight vector higher
Technical docs with codes and IDs	0.6 : 0.4	Exact match is critical — weight BM25 higher
Legal / regulatory documents	0.5 : 0.5	Need both exact section references AND semantic context
FAQ chatbot — short precise answers	0.5 : 0.5	FAQ questions mix exact terms and semantic paraphrase
E-commerce product catalogue	0.7 : 0.3	Product codes and names dominate — BM25 is king

PAKISTAN EG

A Lahore-based legaltech startup built a chatbot for SECP queries. Their pure vector system struggled with Section references. Switching to EnsembleRetriever with weights (0.5, 0.5) improved precision on exact section queries by 34% while maintaining semantic recall for general questions.

Exercise — Build and compare hybrid vs vector-only retrieval (25 min)

1. Create 10 Pakistani document chunks — mix of: a policy PDF, an FAQ, and a regulation. Include some with exact codes/IDs/section numbers.

2. Build two retrievers: a pure Chroma vector_retriever and an EnsembleRetriever(weights=[0.5,0.5]).

3. Write 5 test queries: 3 semantic ("how do I register?") and 2 exact-match ("Section 47(2)" or a specific product code).

4. Run all 5 queries on both retrievers. For each, note: which document ranks #1? Are the results different?

5. Bonus: try weights [0.7, 0.3] vs [0.3, 0.7] on your exact-match queries. Which weight combination performs better?

Day Summary — Monday Week 6

Concept	One-line summary	When to use it
Pure vector fails	Embedding models are poor on IDs, codes, names, numbers	Whenever your corpus has exact-match queries
BM25	Keyword retrieval: rewards rare terms, normalises for doc length	Any time exact-match precision matters
IDF (in BM25)	Rare terms across corpus score higher than common ones	Tuning: larger corpus → better IDF signal
RRF	1/(k+rank) per list, sum across lists — scale-independent fusion	Anytime combining ranked lists from different systems
EnsembleRetriever	BM25 + vector with weights — RRF fusion built in	Default upgrade for any production RAG pipeline
Weight tuning	0.5/0.5 default. Increase BM25 weight for exact-match-heavy docs	Tune based on query type distribution

DO THIS

Tonight: AWS Coursera Week 3 remaining content. Tuesday we add cross-encoder re-ranking — a second stage that re-scores the top results from today's hybrid retriever with a much more powerful (but slower) model.